

# Week 2: benchmark report

## Methodology and evidence

This report uses successful run 20260916\_184848\_404205 from 16 September 2026: **9,394,330 trips** covering 1 January to 31 March 2024. The recorded environment is Spark 4.0.1, `local[*]` and eight shuffle partitions. Machine hardware was not recorded, so these measurements should not be treated as a portable performance target.

Each of 18 cases has one warmup and five timed executions, giving 90 measured executions. Timing includes SQL planning, execution and collection of the complete answer. Correctness checks and plan extraction are outside the timer. Every comparison preserved its answer, with exact counts/nulls and small floating-point tolerances.

The tables show median milliseconds. **Speedup = before / after**; a ratio below 1 means slower. AQE and automatic broadcasting are disabled in the baseline. Experiments run in a fixed order with warm operating-system and JVM caches; Spark caches are cleared between relevant phases. Gold builds and cache population are excluded from query latency.

The copied [metrics](#), [product metrics](#) and [plan files](#) preserve the measured evidence. This run predates extraction of SQL into separate files; all 12 analytical SQL definitions were subsequently checked to match the saved SQL apart from whitespace. These are the recorded timings, not a new run of the refactored code.

## Execution times before and after optimization

### All six queries: Silver versus Gold

Silver calculates from detailed data; Gold reuses saved aggregations to produce the same answers.

| Analysis                 | Silver (ms) | Gold (ms) | Speedup |
|--------------------------|-------------|-----------|---------|
| Monthly zone demand      | 1030.6      | 249.3     | 4.13x   |
| Weather and distance     | 513.7       | 293.0     | 1.75x   |
| Air quality and demand   | 1347.7      | 219.9     | 6.13x   |
| Weather demand variation | 3670.2      | 341.0     | 10.76x  |
| Weekday peak hours       | 1519.2      | 387.1     | 3.92x   |
| Monthly demand trends    | 1311.3      | 282.9     | 4.64x   |

### Focused techniques: one Silver query per experiment

Caching uses weather-distance SQL; AQE uses weather-demand variation. Pruning compares February pickup counts per zone with date bounds alone versus added year/month predicates. Broadcasting compares monthly pickup counts from a taxi/zone join without versus with a broadcast hint.

| Experiment                 | Before (ms) | After (ms) | Speedup |
|----------------------------|-------------|------------|---------|
| Caching                    | 513.7       | 613.0      | 0.84x   |
| AQE and shuffle coalescing | 3670.2      | 3340.0     | 1.10x   |
| Explicit partition filters | 502.6       | 555.3      | 0.91x   |
| Broadcast join             | 4049.4      | 1222.4     | 3.31x   |

Caching and AQE reuse the corresponding Silver baselines. Automatic broadcasting remains disabled during the AQE experiment. The focused experiments isolate techniques; they do not test every technique on every analytical query.

## Analysis of EXPLAIN FORMATTED plans

The evidence folder contains eight plans: one baseline and one optimized plan for each of the four techniques. Both sides use the executed plan from the last measured run, formatted with Spark's `EXPLAIN FORMATTED` representation. Before and after refer to the optimization setting, not the moment of capture. For AQE, the comparison is disabled versus enabled.

**Gold preaggregation.** The weather-distance plans embedded in `metrics.json` show a scan of integrated trips for Silver and a scan of the 786-row `weather_impact_summary` for Gold. Both retain aggregation and exchanges. The main change is the smaller input; separate Gold plan files are unnecessary for the four focused comparisons.

**Caching.** The `before plan` scans integrated Parquet data; the `after plan` contains `Scan In-memory table trips`. The original Parquet scan shown beneath the cache relation describes its source; the active cached scan confirms cache use. Median time nevertheless increased to 613.0 ms, and cache population cost 9.67 seconds. This selected query showed no latency saving to repay that cost.

**AQE.** Compare the `disabled plan` with the `enabled plan`. The enabled plan adds adaptive query stages, `AQEShuffleRead` with Arguments: `coalesced`, and `isFinalPlan=true`. Sort-merge joins remain because automatic broadcasting is disabled. These are actual adaptive changes; the median improved modestly, from 3670.2 to 3340.0 ms.

**Partition pruning.** The `date-only plan` has pushed date filters. The `explicit-filter plan` additionally has `PartitionFilters` for year 2024 and month 2. Both read the same table partitioned by year and month. Its current Delta log contains six files with month-specific date bounds, so date-based skipping is a plausible reason for limited additional benefit. File-read counters were not captured, so equivalent I/O is not established.

**Broadcasting.** The `baseline` uses `SortMergeJoin LeftOuter`, with join-key exchanges and sorts on both inputs. The `hinted plan` uses `BroadcastHashJoin LeftOuter BuildRight` and broadcasts the zone lookup. It removes the large taxi input's join-specific exchange and sort. Aggregation and final ordering still require exchanges; the whole query is not shuffle-free.

## Performance discussion and trade-offs

Gold improved every query, by 1.75x to 10.76x. The four builds took **22.37 seconds** in total and produced **114,319 bytes** of current data files, excluding Delta history and metadata. Build timings exclude subsequent inspection and metadata writes. Repeated analytical use must justify these build and refresh costs.

Broadcasting gave the strongest focused improvement at 3.31x. AQE's 1.10x improvement is modest: baseline samples ranged from 3496.8 to 4307.2 ms and AQE samples from 3293.4 to 3680.0 ms. The ranges overlap. Pruning ranges also overlap (462.8-722.2 versus 499.6-586.7 ms), so the observed slowdown is not evidence that pruning is generally harmful.

The results favor Gold reuse and broadcasting for this workload. Caching and explicit partition predicates changed the plans but did not improve their selected queries. Warm caches, fixed ordering, five samples and local execution limit broader conclusions. Distributed execution or more cities require new measurements of partition sizes, file reads and memory use.